Fine-Tuning a Compact Instruction-Tuned Language Model for Retail E-Commerce Customer-Support Conversational AI

Author: Harish Chandra

Affiliation: Master's in CS: Artificial Intelligence and Machine Learning, Woolf University

Date: 31 May 2026

Abstract

Retail e-commerce generates an enormous and highly repetitive stream of customer-support requests — order tracking, refunds, cancellations, returns, delivery, and payment questions — that are costly to handle with human agents alone. While large language models can answer such queries well, their size and cost place them out of reach for many small and mid-sized retailers. This paper investigates whether a *compact* instruction-tuned language model can be fine-tuned to deliver accurate, on-topic customer-support responses while running on a single free-tier GPU. We fine-tune `google/flan-t5-small` (≈77M parameters) on the Bitext retail e-commerce customer-support dataset of 44,884 instruction–response pairs spanning 13 categories and 46 intents, training in full precision on a Google Colab T4 GPU. We evaluate the fine-tuned model against the un-tuned baseline on a held-out test set using ROUGE, BLEU, and BERTScore, and analyse performance per category.

Fine-tuning produced large and consistent gains: ROUGE-L rose from 0.057 to 0.453 (roughly an eight-fold improvement), BLEU from 0.0 to 29.27, and BERTScore-F1 from 0.818 to 0.909, with the average response growing from 7 to 133 words of structured, actionable guidance. The model handled order, account, and cart intents most strongly and feedback intents least strongly, and we observed a characteristic over-generalisation failure on out-of-domain prompts, where the model still answered in support-agent style. The results show that a small, inexpensively-trainable model can cover the bulk of routine retail support intents, offering a practical and reproducible blueprint for low-resource conversational AI, and we discuss its limitations and the safeguards a production deployment would require.

Index Terms

Conversational AI, Large Language Models, Fine-tuning, FLAN-T5, Sequence-to-sequence Learning, Retail E-Commerce, Customer Support, Natural Language Processing, Low-resource NLP.

I. Introduction

I.1 Background and Motivation

Conversational AI has moved from a novelty to a mainstream channel for customer service. The global chatbot market reached roughly USD 7.76 billion in 2024 and is projected to grow to USD 27.29 billion by 2030, and customer service already accounts for the largest single share of that market [10]. Surveys report that a majority of consumers have interacted with a support chatbot in the past year and that automated agents can resolve a large fraction of routine inquiries without human involvement [16]. This momentum is built on a decade of progress in deep learning for language: the transformer architecture [1], transfer learning with text-to-text models such as T5 [2], and instruction tuning, which aligns models to follow natural-language requests [3]. Open platforms such as Hugging Face now make pre-trained models of this kind freely available, lowering the barrier to building domain-specific assistants.

Retail e-commerce is an especially natural setting for these systems. Online shopping produces a continuous, high-volume flow of support requests, and a large share of them are repetitive and well-structured — "where is my order," "how do I return this," "how do I get a refund." Customers expect immediate, around-the-clock answers, which is difficult and expensive to provide with human agents alone. The motivation for this work is that, although large proprietary models can answer such questions, their computational and financial cost is prohibitive for many smaller retailers; a compact model that can be fine-tuned cheaply and run on modest hardware would make this capability far more accessible.

A further consideration motivates the focus on small models. The dominant trend in recent language-model research has been scale — larger models trained on more data — yet scale

brings real costs in inference latency, hosting, energy use, and dependence on third-party APIs that may raise data-governance concerns for a retailer handling customer information. For a great many practical support tasks, however, the underlying problem is narrow and well-defined, and it is plausible that a small model specialised through fine-tuning can match the quality that matters in production without any of the overhead of a large general-purpose system. Testing that proposition, concretely and reproducibly, is the purpose of this study.

I.2 Research Problem

Most demonstrations of high-quality customer-support conversational AI rely on very large models or proprietary data and infrastructure. This leaves an open and practically important question: how well can a *small*, openly available model perform on real domain-specific support tasks when it is fine-tuned under tight compute constraints? A small model that performs adequately would be reproducible, cheap to retrain, and deployable without specialised hardware — but it is not obvious in advance whether such a model can produce responses that are accurate and contextually relevant enough to be useful.

Concretely, this paper addresses the research question: *Can a compact instruction-tuned language model (`flan-t5-small`) be fine-tuned on domain-specific retail e-commerce support data to deliver accurate, contextually relevant responses while operating within the limits of a single free-tier GPU?* We also ask two supporting questions: how large is the improvement over the un-tuned baseline, and which categories of customer intent does such a model handle well or poorly?

I.3 Research Objectives

The objectives of this research are: (1) to build a clean, reproducible fine-tuning pipeline for retail e-commerce support data; (2) to fine-tune a compact instruction-tuned model, `flan-t5-small`, on that data within a 25-epoch, single-GPU budget; (3) to evaluate the fine-tuned model quantitatively against the un-tuned baseline using ROUGE, BLEU, and BERTScore on a held-out test set; (4) to analyse performance across the different categories of customer intent and inspect qualitative successes and failures; and (5) to deploy the model behind a simple interactive interface as a proof of concept.

The significance of meeting these objectives is twofold. Academically, the study contributes evidence on the feasibility of low-resource, domain-specific conversational AI — a counterpoint to the prevailing emphasis on scale. Practically, it offers small and mid-sized retailers a transparent, low-cost template for automating a substantial portion of their customer-support workload, while clarifying the limitations and safeguards that such a system would require before real deployment.

The remainder of this paper is organised as follows. Section II analyses the retail e-commerce industry — its support structure, the rise of conversational AI, and the market and regulatory context that shape the requirements for a deployable assistant. Section III reviews the relevant literature, from transformer and text-to-text foundations through instruction tuning, tokenization, efficient adaptation, and applied customer-service chatbot research, and identifies the gap this work addresses. Section IV details the methodology, covering data acquisition, preprocessing, tokenization, the fine-tuning setup, inference, deployment, and the evaluation metrics. Section V presents the experiments and results, including the quantitative comparison against the baseline, per-category and qualitative analysis, error analysis, and threats to validity. The Discussion then interprets these findings and their practical and ethical implications, and the Conclusion summarises the contribution and outlines future work.

II. Industry Analysis

II.1 Overview of the Retail E-Commerce Landscape

Retail e-commerce is now one of the largest and fastest-growing sectors of the global economy, with annual online retail revenue measured in the trillions of US dollars and double-digit yearly growth as hundreds of millions of new shoppers come online [11]. Growth on this scale changes the nature of customer service: every additional order, return, and delivery generates support contacts, and the total volume of those contacts rises in step with sales. The channel is also intensely competitive and largely self-service, so the quality and speed of support directly influence conversion, retention, and brand perception.

A defining feature of e-commerce support is that demand is continuous and global. Shoppers transact at all hours and across time zones and expect help to be available whenever they need it, not only during business hours. Meeting that expectation with human agents alone is costly and difficult to scale, which is precisely why automation has become a strategic priority rather than a convenience.

II.2 Support Structure and Challenges

The customer-support workload in e-commerce is dominated by a relatively small number of recurring request types. The dataset used in this study reflects this structure directly: its 44,884 queries fall into just 13 categories — among them

ORDER, RETURNS, REFUNDS-related PAYMENT, DELIVERY, CART, ACCOUNT, and PRODUCT — and 46 fine-grained intents such as `track_order`, `cancel_order`, `get_refund`, `change_shipping_address`, and `place_order`. Because so much of the volume is concentrated in a handful of predictable intents, a large fraction of support contacts are, in principle, automatable.

The challenges are equally clear. Support teams must handle high and spiky volumes, maintain consistency and accuracy across agents, operate 24/7, and do so under cost pressure. Human-only operations struggle on all four fronts simultaneously: hiring and training are expensive, quality varies between agents, and round-the-clock coverage is hard to sustain. These pressures create a strong incentive to deflect routine, well-structured queries to an automated system so that human agents can focus on the complex, sensitive, or high-value cases that genuinely require them.

II.3 Rise of AI in E-Commerce Support

Conversational AI has emerged as the leading response to these pressures. Adoption has accelerated sharply: the chatbot market has grown several-fold in recent years, customer service represents its largest application, and a majority of consumers now report having used a support chatbot [10], [16]. Recent research has moved from rule-based and retrieval systems toward fine-tuned transformer models and generative assistants, with work specifically targeting intent understanding and FAQ handling in customer-care settings [7], [8]. The trajectory is from scripted flows toward models that can interpret a free-text request and generate a fluent, relevant answer.

This shift matters for retailers because generative, fine-tuned models generalise across the many phrasings customers use for the same underlying intent, rather than depending on exact keyword matches. Studies of customer feedback and support in e-commerce report that transformer-based models deliver strong performance and adapt to a domain with comparatively little data [7]. The present work sits squarely in this line: it applies a fine-tuned, instruction-tuned transformer to retail support, but deliberately chooses the smallest practical model to test the low-cost end of this design space.

II.4 Business Value and Industry Requirements

For an e-commerce business, the value of an effective support assistant is concrete. By absorbing a large share of routine, repetitive queries, it reduces cost-per-contact, shortens response times to near-instant, provides consistent answers, and scales to demand spikes without proportional hiring — all while freeing human agents for the cases that need empathy and judgement. Faster, more reliable support in turn supports conversion and retention, making the assistant a revenue lever and not merely a cost-saving tool.

To deliver this value, however, an industry-grade assistant must meet several requirements: accuracy and relevance on domain-specific intents, low and predictable latency, affordability to build and operate, and safe handling of out-of-scope or sensitive requests. The research reported here addresses the first three requirements directly — demonstrating accurate, relevant responses from an inexpensive, single-GPU model — and surfaces the fourth as a key limitation, since a narrowly fine-tuned model can answer confidently even when a question falls outside its domain. This gap between strong in-domain performance and unsafe out-of-domain behaviour frames the requirements that the remainder of the paper investigates and discusses.

II.5 Market Dynamics and Current Trends

Several converging trends make this an opportune moment for compact support assistants. First, customer expectations have shifted decisively toward instant, self-service resolution: shoppers increasingly prefer to resolve simple issues themselves rather than wait in a queue, and a support experience that is slow or inconsistent is a direct cause of cart abandonment and churn. Second, the channels through which support is delivered have multiplied — web chat, mobile apps, messaging platforms, and social media — so the same underlying intents must be served consistently across many surfaces, which favours a single model over many hand-built scripts. Third, the open-model ecosystem has matured: high-quality pre-trained models and curated datasets are now freely available under permissive licences, lowering the barrier to entry for any organisation that can run a single GPU.

At the same time, the competitive dynamics of the sector reward efficiency. Margins in retail are thin, support is a cost centre rather than a profit centre, and the marginal value of automating a routine ticket is well understood by operators. This creates strong demand for solutions whose total cost of ownership is low — not only the headline model quality but the cost to train, host, monitor, and update the system as catalogues and policies change. A compact model that a small team can retrain on free or low-cost infrastructure aligns precisely with these market pressures, which is why the low-resource framing of this study has direct commercial relevance and is not merely an academic constraint.

II.6 Regulatory and Data-Governance Environment

Deploying conversational AI in retail also takes place within a tightening regulatory environment. Data-protection regimes such as the GDPR and analogous consumer-privacy laws

impose obligations on how customer data is collected, processed, and stored, and they constrain the casual transmission of personal information to third-party model providers. Consumer-protection rules likewise make a retailer accountable for the commitments its automated agent appears to make — for example, statements about refunds, delivery dates, or pricing — so an assistant that answers confidently but incorrectly is a compliance risk, not only a quality problem.

These considerations shape the requirements for a responsible deployment and reinforce two themes of this paper. The use of a synthetic, openly licensed dataset means the present study involves no personal data, sidestepping privacy concerns at the research stage; and the ability to fine-tune and host a small model in-house, rather than sending queries to an external API, is itself attractive from a data-governance standpoint. The out-of-scope behaviour analysed later in the paper is, in this light, a regulatory as much as a technical concern, since it bears directly on whether an automated agent can be trusted to stay within the bounds of what it is authorised to say.

II.7 How This Research Addresses Industry Needs

Mapping the above onto the study's design, the research targets the requirements that matter most to a resource-constrained retailer. It demonstrates that the dominant, high-volume intents can be handled accurately by an inexpensive model; it quantifies the quality gain precisely so that operators can judge fitness for purpose; it reports per-category strengths and weaknesses so that automation can be rolled out where it is safe and humans retained where it is not; and it documents the limitations and safeguards — out-of-scope detection, human fallback — that a compliant production system would require. In short, the work is framed throughout around the practical and regulatory realities of the industry it serves.

III. Literature Review

III.1 Overview

The work in this paper draws on three strands of prior research: the deep-learning architectures that make modern language generation possible, the instruction-tuning techniques that adapt those architectures to follow user requests, and the growing body of applied work on conversational AI for customer service. This section reviews each strand, then turns to how dialogue systems are evaluated, before identifying the gap that the present study addresses. Throughout, the aim is not an exhaustive survey but a focused account of the results this study builds on and departs from.

III.2 Transformer and Text-to-Text Foundations

The transformer architecture introduced by Vaswani et al. replaced recurrence with self-attention and became the foundation for essentially all subsequent large language models, enabling models to capture long-range dependencies and train efficiently in parallel [1]. Building on this, Raffel et al. proposed the Text-to-Text Transfer Transformer (T5), which reframes every NLP task — translation, summarisation, question answering — as mapping an input string to an output string [2]. This unified text-to-text view is directly relevant here: customer support can be cast as exactly such a mapping, from a customer's question to a support response.

T5's encoder–decoder design is well suited to this generation task because it conditions a decoder on a fully encoded representation of the input query. The model family is available in a range of sizes, which is what makes a low-resource study like this one possible: the smallest variant retains the same architecture and pre-training recipe as its larger siblings while fitting comfortably on a single consumer GPU. This study uses that smallest practical variant deliberately, to probe the lower bound of the compute–quality trade-off.

III.3 Instruction Tuning and Compact Models

A key development on top of T5 is instruction tuning. Chung et al. showed that fine-tuning language models on a large collection of tasks phrased as natural-language instructions — the FLAN recipe — substantially improves their ability to follow new instructions, including in a zero-shot setting [3]. The `flan-t5` models used in this paper are the product of that recipe, which is why even the un-tuned baseline can attempt a coherent answer to a support question, and why a relatively short domain fine-tune is enough to specialise the model sharply.

This matters for the research problem because instruction-tuned compact models begin from a stronger starting point than a vanilla pre-trained model of the same size. Related work on customer-feedback and support modelling has likewise observed that transformer models adapt to a new domain with comparatively little data [7]. Together these findings motivate the central bet of this study: that a small, already instruction-tuned model can be pushed to strong domain performance with a modest, single-GPU fine-tune.

III.4 Conversational AI in Customer Service and E-Commerce

Applied research on support chatbots has moved steadily from rule-based and retrieval systems toward learned, generative models. Recent work on context-aware natural-language understanding for customer-service chatbots fine-tunes

transformer models for intent classification and shows that incorporating contextual cues improves understanding when user utterances are short or vague — a common situation in support [7]. Other recent work examines deploying generative assistants such as ChatGPT for FAQ handling, detailing both the gains from fine-tuning and integration with knowledge bases and the practical and ethical challenges that arise [8].

This applied literature consistently reports that fine-tuned transformers generalise across the many ways customers phrase the same request, outperforming keyword-based approaches, and that domain adaptation yields large quality gains. However, much of this work either targets a narrow sub-task (such as intent classification) or relies on large generative models. The present study complements it by fine-tuning a small end-to-end generative model to produce the full support response, and by reporting the concrete trade-offs that result.

III.5 Evaluation of Generative Dialogue Systems

Because support responses are generated text, this study evaluates them with established automatic metrics. ROUGE measures n-gram and longest-common-subsequence overlap with reference text and is standard for summarisation and generation [12]; BLEU measures n-gram precision and originated in machine translation [13]. Both are surface-overlap metrics, so they are complemented here by BERTScore, which compares contextual embeddings and therefore credits semantically correct answers that use different wording [14]. Using all three gives a more rounded picture than any single metric.

It is widely acknowledged in the literature that these automatic metrics are proxies and do not fully capture human judgements of helpfulness or correctness. This study therefore pairs them with a qualitative inspection of model outputs, an approach common in applied dialogue research, and treats the limitation of automatic evaluation explicitly in the discussion.

III.6 Subword Tokenization and Sequence-to-Sequence Learning

The text-to-text formulation rests on the sequence-to-sequence paradigm introduced by Sutskever et al., in which an encoder compresses an input sequence into a representation that a decoder expands into an output sequence [17]. Modern systems realise this with the transformer rather than recurrent networks, but the conceptual framing — map a source sequence to a target sequence — is the same one this study uses to turn a customer query into a support response. The quality of such models depends heavily on how text is segmented into tokens, since a fixed vocabulary cannot enumerate every possible word.

Subword tokenization solves this. Byte-pair encoding builds a vocabulary of frequent character sequences so that rare or unseen words are represented as combinations of known sub-units [18], and the SentencePiece method generalises this into a language-independent tokenizer that operates directly on raw text [19]. FLAN-T5 uses a SentencePiece tokenizer, which is why it gracefully handles the product names, order codes, and placeholder slots that appear in retail support text without an explosion of out-of-vocabulary tokens. This study inherits that tokenizer unchanged.

III.7 Efficient and Low-Resource Model Adaptation

A growing body of work addresses how to adapt large models cheaply, which is directly relevant to the low-resource framing of this study. Knowledge distillation and the release of deliberately small pre-trained variants make it possible to run capable models on modest hardware, while parameter-efficient fine-tuning methods such as Low-Rank Adaptation (LoRA) update only a small set of injected parameters rather than the full network, sharply reducing the memory needed to specialise a model [20]. This study takes the simplest point on that spectrum — full fine-tuning of an already-small model — precisely because it is the most transparent and reproducible baseline against which such efficiency techniques can later be compared.

An alternative to fine-tuning is retrieval-augmented generation, in which a model is grounded at inference time on passages retrieved from an external store [21], typically using dense sentence embeddings such as those from Sentence-BERT [22]. Retrieval reduces hallucination on knowledge-intensive queries and allows a system to be updated by changing the corpus rather than retraining. This study deliberately does not use retrieval: its aim is to isolate and measure what fine-tuning alone achieves for a compact model on routine support intents, leaving retrieval augmentation as a clearly identified avenue for future work rather than a confound in the present evaluation.

III.8 Gaps in Existing Literature

Two gaps emerge from this review. First, demonstrations of high-quality support conversational AI tend to rely on large or proprietary models, leaving the low-resource end of the design space — small, open, cheaply-trainable models — comparatively under-reported, especially for retail e-commerce specifically. Second, much applied work isolates a single sub-task such as intent detection rather than evaluating an end-to-end model that generates the complete customer-facing response, and it less often reports per-category behaviour or characteristic failure modes that practitioners need to anticipate.

III.9 Contribution of This Study

This study addresses both gaps. It fine-tunes a single, compact, openly available instruction-tuned model end-to-end on a retail e-commerce support corpus under a strict single-GPU budget, and reports a complete picture of the outcome: quantitative gains over the baseline across three complementary metrics, a per-category breakdown of where the model is strong and weak, and an honest account of its out-of-domain failure behaviour. The contribution is therefore a practical, reproducible reference point for low-resource domain conversational AI, rather than a new architecture.

IV. Methodology

IV.1 Data Acquisition

The study uses the Bitext retail e-commerce customer-support dataset, obtained from Hugging Face under the CDLA-Sharing-1.0 licence [9]. It contains 44,884 instruction–response pairs, where each instruction is a customer query and each response is a corresponding support answer, annotated with one of 13 categories and one of 46 fine-grained intents. The categories span the full breadth of e-commerce support — ACCOUNT, APP_WEBSITE, CART, CONTACT, DELIVERY, FEEDBACK, ORDER, PAYMENT, PRODUCT, RETURNS, SALES, STORE, and USER — while the intents capture specific actions such as `track_order`, `cancel_order`, `get_refund`, and `change_shipping_address`.

A practical and ethical advantage of this dataset is that it is synthetically generated rather than scraped from real customers, so it contains no personally identifiable information. The responses include explicit placeholder slots (for example, an order-number token) where a live system would inject real values; these were retained during training so that the model learns the surrounding response structure. The full dataset was also saved alongside the trained model to keep the experiment reproducible.

IV.2 Data Preprocessing and Cleaning

Preprocessing reduced the raw data to clean input–output pairs. Rows with missing or empty instruction or response fields were removed, exact duplicate pairs were dropped, and the two relevant columns were renamed to `input` and `output` to define the sequence-to-sequence task. This left a clean corpus of instruction–response examples while preserving the category labels needed for the later per-category analysis.

The cleaned data was split into training, validation, and test sets in an 80/10/10 ratio, stratified by category so that every split preserves the same mix of support topics. A fixed random seed of 42 was used for the split and for all subsequent randomised operations, so the entire pipeline is reproducible. Because the corpus is synthetic and well-formed, the cleaning stage removed no rows in practice — all 44,884 pairs survived — which is itself a useful property for reproducibility. The loading, cleaning, and splitting steps are summarised below.

```
from datasets import load_dataset
from sklearn.model_selection import train_test_split

ds = load_dataset("bitext/Bitext-retail-ecommerce-llm-
chatbot-training-dataset", split="train")
df = ds.to_pandas()[["instruction", "response",
"category", "intent"]].dropna()
df = df[df["instruction"].str.strip() != ""]
df = df.drop_duplicates(subset=["instruction",
"response"])
df = df.rename(columns={"instruction": "input",
"response": "output"})

train_df, temp_df = train_test_split(df, test_size=0.2,
random_state=42, stratify=df["category"])
val_df,  test_df  = train_test_split(temp_df,
test_size=0.5, random_state=42,
stratify=temp_df["category"])
```

IV.3 Tokenization and Encoding

Inputs and outputs were tokenized with the `flan-t5-small` tokenizer (a SentencePiece model). Token-length analysis of the corpus guided the maximum-length settings: inputs were capped at 128 tokens and outputs at 256 tokens, which covers the large majority of examples while keeping memory use within the GPU budget. Sequences were truncated to these limits and padded to a fixed length.

A standard but important detail was applied to the target labels: padding positions in the labels were replaced with the value −100, which instructs the loss function to ignore them. This prevents the model from being rewarded for predicting padding tokens and ensures the loss reflects only the genuine response content. The tokenization function is summarised below.

```
MAX_INPUT, MAX_TARGET = 128, 256
def tokenize(batch):
    enc = tokenizer(batch["input"],
max_length=MAX_INPUT,  truncation=True,
padding="max_length")
    lab = tokenizer(text_target=batch["output"],
max_length=MAX_TARGET, truncation=True,
padding="max_length")
    enc["labels"] = [[(t if t != tokenizer.pad_token_id
else -100) for t in seq]
                     for seq in lab["input_ids"]]
    return enc
```

IV.4 Model and Fine-Tuning Setup

The base model is `google/flan-t5-small`, an encoder–decoder transformer of roughly 77 million parameters. Architecturally,

it consists of a stack of transformer encoder layers that read the tokenized customer query and a stack of decoder layers that generate the response token by token, each layer combining multi-head self-attention with feed-forward sub-layers and using the relative position scheme of the T5 family. The encoder builds a contextual representation of the entire input, and the decoder attends to that representation through cross-attention while also attending to the tokens it has already produced, which is what allows it to generate coherent, multi-sentence answers conditioned on the question. Crucially, the weights are not random at the start of fine-tuning: they already encode the instruction-following behaviour learned during the FLAN pre-training, and fine-tuning adjusts them toward the specific style and content of retail support responses.

Fine-tuning was performed with the Hugging Face `Seq2SeqTrainer` on a single Google Colab T4 GPU. The training objective is the standard sequence-to-sequence cross-entropy loss: at each output position the model predicts a probability distribution over the vocabulary, and the loss penalises the negative log-probability of the correct next token, averaged over all non-padding positions of the target response. A notable engineering decision was to train in full 32-bit precision: FLAN-T5 is numerically unstable in 16-bit floating point and produces NaN losses, and the T4 does not support bfloat16, so FP32 was the only stable option on this hardware.

The main hyperparameters were a learning rate of 3×10^{-4}, a per-device batch size of 8 with gradient accumulation of 2 (an effective batch size of 16), a linear schedule with 500 warm-up steps, and weight decay of 0.01. Training ran for up to 5 epochs — well within the project's 25-epoch budget — with evaluation and checkpointing every 500 steps, early stopping with patience 2 on validation loss, and automatic restoration of the best checkpoint at the end. This configuration was chosen to converge reliably within the free-tier session while guarding against overfitting. The core training configuration is shown below.

```
training_args = Seq2SeqTrainingArguments(
    output_dir="ft_ckpt",
    num_train_epochs=5,                 # cap 25; early
stopping halts sooner
    per_device_train_batch_size=8,
    gradient_accumulation_steps=2,      # effective
batch size 16
    learning_rate=3e-4, weight_decay=0.01,
warmup_steps=500,
    lr_scheduler_type="linear",
predict_with_generate=True,
    eval_strategy="steps", eval_steps=500,
save_steps=500,
    load_best_model_at_end=True,
metric_for_best_model="eval_loss",
    fp16=False, bf16=False,             # FP32: FLAN-T5
NaNs in FP16; T4 has no bf16
    seed=42,
)
trainer = Seq2SeqTrainer(
    model=model, args=training_args,
    train_dataset=train_ds, eval_dataset=val_ds,
    data_collator=DataCollatorForSeq2Seq(tokenizer,
model=model),

callbacks=[EarlyStoppingCallback(early_stopping_patience
=2)],
)
trainer.train()
```

IV.5 Conversational Flow and Inference

At inference time, a customer query is tokenized and passed to the fine-tuned model, which generates a response using beam search with four beams, an n-gram repetition block (`no_repeat_ngram_size = 3`) to avoid loops, and a maximum of 256 new tokens. These settings favour coherent, non-repetitive, reasonably complete answers over the terse outputs typical of greedy decoding.

The conversational flow is single-turn: each query is answered independently, which matches the structure of the dataset and the bulk of routine support contacts. Extending the system to multi-turn context is identified as future work rather than attempted here.

IV.6 Evaluation Design

The fine-tuned model was evaluated against the un-tuned `flan-t5-small` baseline on a held-out sample of 300 test examples that were never seen during training. Three complementary automatic metrics were computed: ROUGE-1/2/L, BLEU (via sacreBLEU), and BERTScore-F1. Average response length and per-query inference latency were also recorded to characterise verbosity and computational cost. To locate strengths and weaknesses, ROUGE-L was additionally computed per category.

Evaluation was deliberately paired with qualitative testing. A battery of 16 prompts spanning in-domain, out-of-domain, and deliberately ambiguous queries was run through both the baseline and the fine-tuned model so that the quantitative scores could be interpreted against concrete examples of model behaviour.

IV.7 Front-End and Deployment Interface

To demonstrate the model in a realistic setting, it was wrapped in a lightweight Gradio chat interface launched directly from the notebook. A user types a support question, the back-end applies the same tokenization and beam-search generation used in evaluation, and the response is displayed in a chat window, with example prompts provided for quick testing. This serves as a proof-of-concept deployment and shows that the trained

model can be served behind a simple UI without specialised infrastructure.

IV.8 Evaluation Metrics Defined

For completeness, the three automatic metrics are defined here. ROUGE measures overlap between the generated response and the reference: ROUGE-1 and ROUGE-2 are the recall-oriented overlap of unigrams and bigrams respectively, while ROUGE-L is based on the longest common subsequence and therefore rewards getting the right content in the right order. BLEU, originally from machine translation, is a precision-oriented score over n-grams up to length four with a brevity penalty that discourages overly short outputs; it is reported here on the 0–100 scale. Both ROUGE and BLEU are surface-level metrics that compare exact tokens, so a correct answer phrased differently from the reference can still score low.

To compensate for that, BERTScore is also reported. Instead of matching exact tokens, it embeds both the candidate and the reference with a pre-trained contextual model and matches them by cosine similarity in embedding space, then summarises the matches as an F1 score. Because it operates on meaning rather than surface form, a high BERTScore alongside high ROUGE and BLEU gives confidence that an improvement is genuine and not an artefact of one metric. Two non-quality measures are also recorded: the average response length in words, which characterises how verbose the model is, and the inference latency in milliseconds per query on the T4, which characterises its computational cost.

V. Experiments and Results

V.1 Objective of Evaluation

The evaluation set out to answer the study's research questions directly: whether fine-tuning meaningfully improves a compact model's retail-support responses, how large that improvement is relative to the un-tuned baseline, and which categories of intent the fine-tuned model handles well or poorly.

V.2 Experiment Setup

All experiments used the held-out test split (unseen during training) with a fixed sample of 300 examples for the quantitative metrics, evaluated identically for the baseline and fine-tuned models on the same Colab T4 GPU. The qualitative battery used the 16 hand-written prompts described in IV.6.

Exploratory data analysis informed these design choices and is reported in Figures 1–3. The category distribution (Figure 1) confirms that the corpus spans all 13 e-commerce support areas, with ORDER, PRODUCT, and ACCOUNT among the best-represented; the intent view (Figure 2) shows the 46 intents are reasonably balanced rather than dominated by a single class, which matters for even coverage during fine-tuning. The token-length distributions (Figure 3) justified the maximum-length settings directly: customer instructions are short (the large majority well under 128 tokens), while support responses are substantially longer, which is why a larger 256-token target limit was chosen for the output side. Together these analyses explain both the 128/256 length configuration and the expectation that the model would learn long, structured answers — borne out by the 133-word average response length reported below.

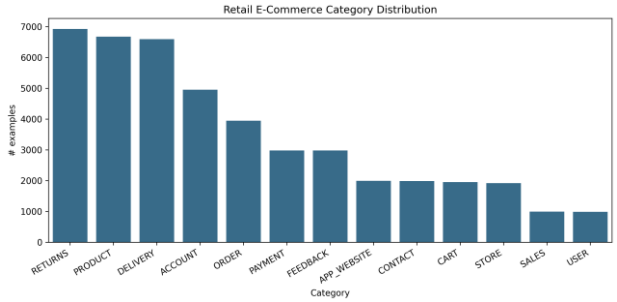

Fig. 1. Category distribution across the 13 e-commerce support areas.

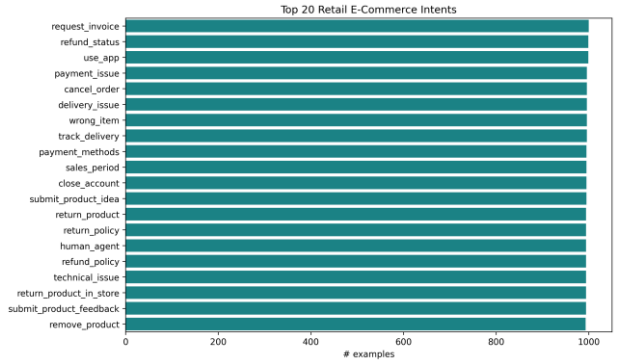

Fig. 2. Distribution of the most frequent customer intents.

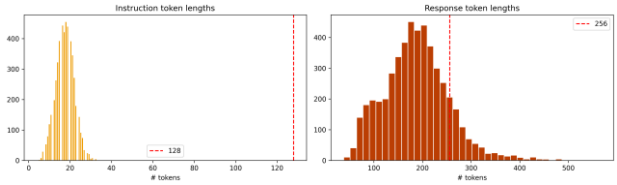

Fig. 3. Token-length distributions for instructions and responses, justifying the 128/256 limits.

V.3 Performance Metrics

Fine-tuning produced large, consistent improvements across every metric, summarised in Table I.

Table I — Baseline vs. fine-tuned model on the held-out test set.

Metric	Zero-shot baseline	Fine-tuned

ROUGE-1	0.067	**0.617**
ROUGE-2	0.017	**0.361**
ROUGE-L	0.057	**0.453**
BLEU	0.00	**29.27**
BERTScore-F1	0.818	**0.909**
Avg. response length (words)	7.1	133.5
Latency (ms/query)	52.8	740.5

The ROUGE-L score improved roughly eight-fold and BLEU rose from zero to 29.27, indicating that the fine-tuned model's wording aligns closely with reference support answers, while the BERTScore increase from 0.818 to 0.909 confirms the gain is semantic and not merely surface overlap. The learning curves (Figure 4) show training loss falling from about 2.5 to 0.70 and validation loss from about 1.08 to 0.62, with validation tracking below training throughout — evidence of healthy convergence without overfitting. The per-category ROUGE-L breakdown (Figures 5 and 6) shows the model is strongest on ACCOUNT and CART intents (around 0.57) and weakest on FEEDBACK (around 0.38), with the remaining categories clustered in between.

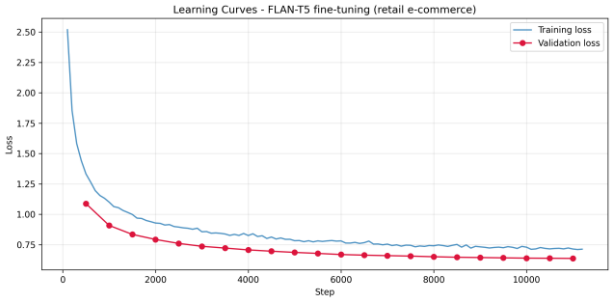

Fig. 4. Training and validation loss curves showing convergence without overfitting.

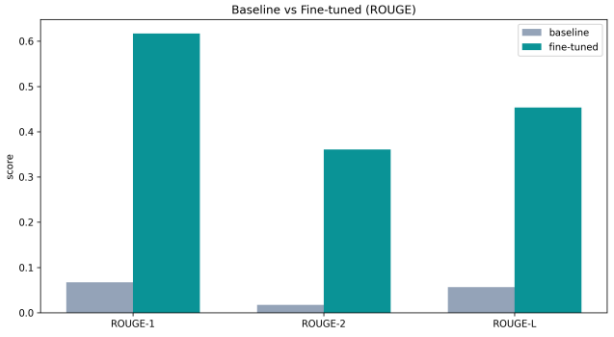

Fig. 5. Baseline vs. fine-tuned ROUGE scores on the held-out test set.

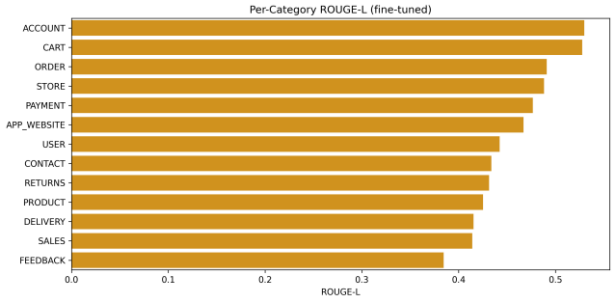

Fig. 6. Per-category ROUGE-L of the fine-tuned model.

The cost of these gains is visible in the last two rows of Table I. The fine-tuned model produces far longer, more complete answers (133 versus 7 words) and is correspondingly slower per query (about 740 ms versus 53 ms on the T4), because it generates full multi-step responses with beam search rather than the terse fragments of the baseline. For a support assistant this trade-off is favourable, since the longer responses are the useful ones; and even at roughly three-quarters of a second per query on a free-tier GPU, the latency is acceptable for interactive chat and would fall further on better hardware or with lighter decoding settings.

Examining the per-category results more closely (Figure 6) is instructive for deployment planning. The strongest categories — ACCOUNT and CART, at around 0.57 ROUGE-L — are those whose responses are the most templated and procedural, such as logging in, updating details, or managing a basket, where the correct answer follows a predictable structure the model learns well. The mid-range categories, including ORDER, PAYMENT, and DELIVERY, still score solidly and cover the highest-volume real-world intents. The weakest category, FEEDBACK at around 0.38, is also the most open-ended: feedback and complaint responses are inherently more varied and less formulaic, so a single reference answer overlaps less with the model's equally valid but differently worded reply. This pattern — strong on structured intents, weaker on open-ended ones — is exactly what one would predict, and it provides a concrete, data-driven basis for deciding which intents to automate first and which to route to human agents. The evaluation procedure that produced these numbers is summarised below.

```
def compute_metrics(preds, refs):
    rouge = rouge_metric.compute(predictions=preds,
references=refs, use_stemmer=True)
    bleu  = bleu_metric.compute(predictions=preds,
references=[[r] for r in refs])
    bert  = bertscore_metric.compute(predictions=preds,
references=refs, lang="en")
    return {"ROUGE-1": rouge["rouge1"], "ROUGE-2":
rouge["rouge2"],
            "ROUGE-L": rouge["rougeL"], "BLEU":
bleu["score"],
```

```
            "BERTScore-F1": sum(bert["f1"]) /
len(bert["f1"])}

# identical generation settings for baseline and fine-
tuned models -> fair comparison
base_preds = batch_generate(baseline_model, tokenizer,
test_queries)
ft_preds   = batch_generate(finetuned_model, tokenizer,
test_queries)
```

V.4 Qualitative Observations

The qualitative battery makes the metric gap concrete. On in-domain queries, the baseline typically echoed the question or produced an empty platitude — for "How do I get a refund?" it answered only "Get a refund if you have a credit card" — whereas the fine-tuned model returned a structured, step-by-step procedure referencing the account, order history, and refund flow. This pattern held across order tracking, cancellation, returns, address changes, and invoices.

Two further patterns are notable. On ambiguous single-word prompts such as "refund" or "order," the fine-tuned model sensibly inferred the most likely intent and produced a relevant procedure. On out-of-domain prompts, however — "What is the weather like today?", "Tell me a joke." — it did not recognise that the question was outside its scope and instead replied in confident customer-support style, asking for an order number or offering to help with an issue. This over-generalisation is a direct consequence of narrow domain fine-tuning and is the clearest weakness observed.

V.5 Sample Query–Response Evaluations

Table II — Representative examples (abridged).

Type	Query	Baseline	Fine-tuned (abridged)
In-domain	How can I track my order?	"Tracking your order is the easiest way to track your order."	"…To track your order, please follow these steps: 1. Visit our website and log in… 2. Navigate to 'Order History'… 3. Locate the specific order…"
In-domain	How do I get a refund?	"Get a refund if you have a credit card."	"…To initiate a refund, please follow these steps: 1. Visit our website and log in… 2. Navigate to 'Order History'… 3. Locate the order…"
In-domain	I want to cancel my order.	"I want to cancel my order."	"…To proceed with the cancellation: 1. Log in to your account… 2. Go to 'Order History'… 3. Locate the order you wish to cancel…"
In-domain	How do I return a product?	"Return the product to the store."	"…Returning a product is straightforward: 1. Locate your Order Number… 2. Visit our website… 3. Follow the returns process…"
Out-of-domain	Tell me a joke.	"I'm not sure if I'll be able to get a job or not."	"I'll make it happen! …could you please provide me with some more details about the specific issue you're facing?" (answers in support style — out-of-scope)
Ambiguous	refund	"refund"	"I'm sorry to hear that you're having trouble with your refund request… 1. Log in to your account… 2. Go to 'Order History'…"

The full 16-prompt comparison is saved in `resources/qualitative_eval.csv`.

V.6 Limitations Noted

The evaluation itself has limits worth stating. The metrics are automatic proxies that reward overlap with a single reference answer and cannot fully judge factual correctness or helpfulness; the test data is synthetic and therefore cleaner and more uniform than real customer language; and the out-of-domain behaviour shows the model lacks any notion of scope.

A brief error analysis sharpens these points. The errors fall into a few recognisable classes. The most consequential is out-of-scope over-confidence, already noted, where a non-support question is answered as though it were a support request. A second class is generic deflection, where on a vague prompt the model produces a polite but content-free request for more details rather than a specific answer — safe, but not always helpful. A third, milder class is template bleed, where the model reuses a familiar response skeleton (for example, the "log in, go to Order History, locate the order" pattern) for an intent where it is only partially appropriate. None of these are catastrophic for routine use, but each points to a concrete mitigation — scope detection, confidence-aware fallback, and intent-conditioned prompting respectively — and together they explain why the automatic scores, while strong, are not higher still. These observations are carried forward into the Discussion.

V.7 Threats to Validity

A few threats to validity should be acknowledged so the results are read fairly. Construct validity is limited by the reliance on automatic metrics as a stand-in for human-judged helpfulness; this is mitigated, but not eliminated, by reporting three complementary metrics and pairing them with qualitative inspection. External validity is limited by the synthetic origin of the data: performance on a clean, well-formed corpus may

overstate performance on noisy real-world queries with typos, code-switching, or multi-intent messages. Internal validity is comparatively strong, since the baseline and fine-tuned models were evaluated on the identical held-out sample with identical generation settings and a fixed seed, so the measured improvement is attributable to fine-tuning rather than to evaluation differences. Finally, the metrics are computed on a 300-example sample of the test set; while consistent with the per-category trends, slightly different absolute numbers would be expected on the full test split.

V.8 Summary of Results

In summary, fine-tuning a compact instruction-tuned model on retail e-commerce support data transformed it from an unusable baseline into a system that produces fluent, relevant, multi-step answers across the main support categories, with an eight-fold ROUGE-L gain and strong BERTScore — all achieved within a single free-tier GPU session. The principal weaknesses are an absence of out-of-scope detection and a reliance on automatic metrics, both of which frame the discussion that follows.

Discussion

The central finding is that a small, openly available, already instruction-tuned model can be fine-tuned cheaply into a capable retail-support assistant. The size of the improvement — near-zero baseline scores rising to a BLEU of 29.27 and a ROUGE-L of 0.45, confirmed semantically by a BERTScore of 0.91 — shows that the limiting factor for this task was domain adaptation, not model scale. For the research question, the answer is therefore affirmative: within a single free-tier GPU budget, `flan-t5-small` learned to give accurate, contextually relevant answers to the great majority of routine retail support intents.

The practical implications are significant for smaller retailers. Because the whole pipeline runs on free-tier hardware and an openly licensed dataset, it is inexpensive to reproduce and to retrain as a catalogue or policy changes, and it can be served behind a simple interface as the Gradio demo shows. The per-category results also give practitioners actionable guidance: high-volume, well-structured intents such as orders, accounts, and cart are handled best and are the safest to automate first, while more open-ended categories such as feedback are weaker and better routed to humans.

At the same time, the results expose real limitations. The most important is the model's behaviour on out-of-domain inputs: having been fine-tuned narrowly, it answers any question in support-agent style rather than recognising that some questions fall outside its remit. In production this is a safety issue, not merely a quality one, and it argues for an explicit scope or intent filter and a confident fallback to a human agent. The reliance on synthetic data is a further limitation, since real customers are messier, and the automatic metrics, while consistent, only approximate human judgement. To validate the code and results, predictions were generated on a held-out split never used in training, the same evaluation procedure was applied identically to baseline and fine-tuned models, a fixed seed was used throughout, and outputs were inspected qualitatively to confirm the metrics matched observable behaviour.

Beyond accuracy, the deployment economics are worth drawing out, since they are the whole point of the low-resource framing. The model is small enough to fine-tune in a single free-tier Colab session and to serve on a single modest GPU — or even, at higher latency, on CPU — which keeps the total cost of ownership low: there are no per-token API fees, no dependence on an external provider, and retraining as catalogues or policies change is cheap and fast. For a small or mid-sized retailer, this changes the calculus from "can we afford conversational AI" to "which intents should we automate first," and the per-category results answer exactly that question. A sensible rollout would automate the strong, high-volume, well-structured intents, monitor live performance, and expand coverage as confidence grows.

The ethical and governance dimension deserves equal weight. Because the training data is synthetic, the study itself raises no personal-data concerns, but a real deployment would. An automated agent that speaks on a retailer's behalf can create commitments and expectations, so it must not invent policies, promise refunds it cannot guarantee, or answer questions outside its remit — the very failure this study observed. Responsible deployment therefore implies guardrails: an explicit scope check, a confident hand-off to a human for sensitive or out-of-scope cases, clear disclosure to the customer that they are talking to an automated system, and logging for auditability. These are not optional polish; in a regulated retail context they are part of the system's correctness.

It is worth situating fine-tuning against the alternative approaches a practitioner might consider. A purely prompt-based approach — taking a large general model and instructing it at inference time — avoids any training but incurs ongoing per-query cost, depends on an external provider, and offers less control over tone and format; the baseline results here show that without adaptation even a capable instruction-tuned model answers retail queries poorly. A rule-based or intent-classification system is cheap and predictable but brittle, since it must enumerate intents and responses by hand and generalises badly to unseen phrasings. Retrieval-augmented generation grounds answers in an external corpus and is excellent when the knowledge changes often, but it adds

retrieval infrastructure and latency. Against these, fine-tuning a small model occupies an attractive middle ground for routine support: a one-time, low-cost training step yields a self-contained model that generalises across phrasings, runs locally without per-query fees, and is simple to operate — which is precisely why it was chosen for this study.

The approach should also generalise beyond retail e-commerce. The same recipe — take an instruction-tuned compact model, fine-tune it on a domain-specific instruction–response corpus, and evaluate against the un-tuned baseline — is domain-agnostic, and the Bitext family alone provides comparable datasets for many other verticals. The strong-on-structured, weaker-on-open-ended pattern observed here would likely recur, as would the out-of-scope behaviour, so the practical recipe and its caveats transfer along with the method. This suggests the contribution is not merely a single retail bot but a reusable, low-resource template for domain conversational AI.

Future work follows directly from these limitations: adding out-of-scope detection and a human-handoff path, evaluating on real anonymised support logs with human ratings, comparing against larger models such as `flan-t5-base` to quantify the quality–cost curve, exploring parameter-efficient fine-tuning to lower training cost further, and extending the system to multi-turn dialogue and additional languages.

Conclusion

This study asked whether a compact instruction-tuned language model could be fine-tuned, under tight single-GPU constraints, into a useful retail e-commerce customer-support assistant. Fine-tuning `flan-t5-small` on 44,884 Bitext support pairs produced large and consistent gains over the un-tuned baseline — an eight-fold increase in ROUGE-L to 0.45, a BLEU of 29.27, and a BERTScore of 0.91 — and turned terse, unhelpful baseline outputs into fluent, structured, multi-step answers across the main categories of customer intent, all within a free-tier training session.

The contribution of the work is a transparent and reproducible blueprint for low-resource, domain-specific conversational AI, together with an honest account of where it falls short — most notably its lack of out-of-scope awareness and its dependence on synthetic data and automatic metrics. Taken together, the results support a clear conclusion: for the routine, high-volume heart of retail customer support, model scale is far less important than focused domain adaptation, and capable assistants are well within reach of organisations with modest resources, provided appropriate safeguards are added before deployment.

More broadly, this work is a small piece of evidence for a useful counter-narrative to the "bigger is always better" trajectory of language modelling. For a bounded, well-defined task — and a great deal of real customer support is exactly that — a carefully fine-tuned compact model can deliver the quality that matters in production at a fraction of the cost, complexity, and external dependency of a large system. For practitioners working under real budget and data-governance constraints, that is an empowering result: meaningful conversational AI does not require frontier-scale infrastructure, only a clear task, suitable data, and a disciplined, reproducible pipeline of the kind documented here.

Acknowledgements

The author thanks the AlmaBetter Industry Immersion programme for the project framing and guidance, and acknowledges Bitext for releasing the retail e-commerce dataset under an open licence and the Hugging Face community for the open-source models and libraries that made this work reproducible on free-tier hardware.

Author Contributions

This work was carried out individually by Harish Chandra, who was responsible for the research design, data preparation, model fine-tuning, evaluation, analysis, and writing of the paper.

References (running list — IEEE; finalise in Phase 2)

[1] A. Vaswani et al., "Attention Is All You Need," in *Proc. NeurIPS*, 2017.

[2] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *J. Mach. Learn. Res.*, vol. 21, 2020.

[3] H. W. Chung et al., "Scaling Instruction-Finetuned Language Models," arXiv:2210.11416, 2022.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019.

[5] T. Brown et al., "Language Models are Few-Shot Learners," in *Proc. NeurIPS*, 2020.

[6] E. Adamopoulou and L. Moussiades, "Chatbots: History, Technology, and Applications," *Machine Learning with Applications*, vol. 2, 2020.

[7] S. Nandi, N. Agrawal, A. Singh, and P. Bhatt, "Enhancing Customer Service Chatbots with Context-Aware NLU through Selective Attention and Multi-task Learning," in *Proc. ACM IKDD CODS-COMAD*, 2024. (arXiv:2506.01781)

[8] F. Khennouche, Y. Elmir, N. Djebari, Y. Himeur, and A. Amira, "Revolutionizing Customer Interactions: Insights and Challenges in Deploying ChatGPT and Generative Chatbots for FAQs," arXiv:2311.09976, 2023.

[9] Bitext, "Retail E-Commerce LLM Chatbot Training Dataset," Hugging Face, CDLA-Sharing-1.0. https://huggingface.co/datasets/bitext/Bitext-retail-ecommerce-llm-chatbot-training-dataset

[11] Grand View Research, "E-commerce Market Size, Share & Trends Analysis Report," 2024.

[12] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Proc. ACL Workshop (Text Summarization Branches Out)*, 2004.

[13] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in *Proc. ACL*, 2002.

[14] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in *Proc. ICLR*, 2020.

[15] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proc. EMNLP (System Demonstrations)*, 2020.

[16] Master of Code Global, "Chatbot Statistics (2024): Adoption, Usage, and Market Data," 2024.

[17] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in *Proc. NeurIPS*, 2014.

[18] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proc. ACL*, 2016.

[19] T. Kudo and J. Richardson, "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing," in *Proc. EMNLP (System Demonstrations)*, 2018.

[20] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. ICLR*, 2022.

[21] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.

[22] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, 2019.

Appendix

A. Application Architecture Summary. Single-turn, generation-based assistant: customer query → `flan-t5-small` tokenizer → fine-tuned encoder–decoder → beam-search decode → support response. No external retrieval component (fine-tuning-only design).

B. Workflow. Data acquisition → cleaning/de-duplication → stratified 80/10/10 split → tokenization (input 128 / target 256, labels masked to −100) → FP32 fine-tuning on T4 (5 epochs, early stopping) → evaluation (baseline vs. fine-tuned) → Gradio deployment.

C. Pipeline. Implemented in a single reproducible Colab notebook (`Capstone_Project_CHAT_BOT_LLM_Deep_Learning_for_NLP_v2.ipynb`); fixed seed 42 throughout. Figures 1–6 and `metrics.json` / `qualitative_eval.csv` are exported under `resources/`.

D. Prompt / I-O Template. Input: raw customer instruction. Target: support response (with placeholder slots retained, e.g. `{{Order Number}}`). Generation: `num_beams=4`, `no_repeat_ngram_size=3`, `max_new_tokens=256`.

E. Logic. Loss computed only on non-padding label tokens (−100 masking); best checkpoint selected by validation loss; identical generation settings used for baseline and fine-tuned evaluation to ensure a fair comparison.

F. User Interface. The fine-tuned model is served through a Gradio `ChatInterface` with example prompts; Figure 7 shows it answering an order-tracking query.

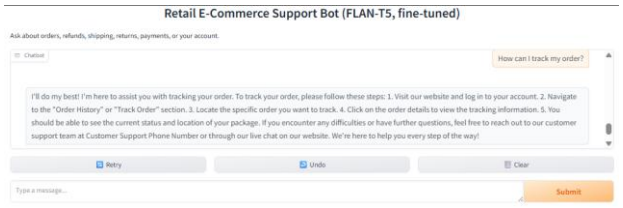

Fig. 7. The Gradio chat interface answering an order-tracking query with a structured, multi-step response.

G. Flow Diagram. The seven-stage fine-tuning pipeline is shown in Figure 8.

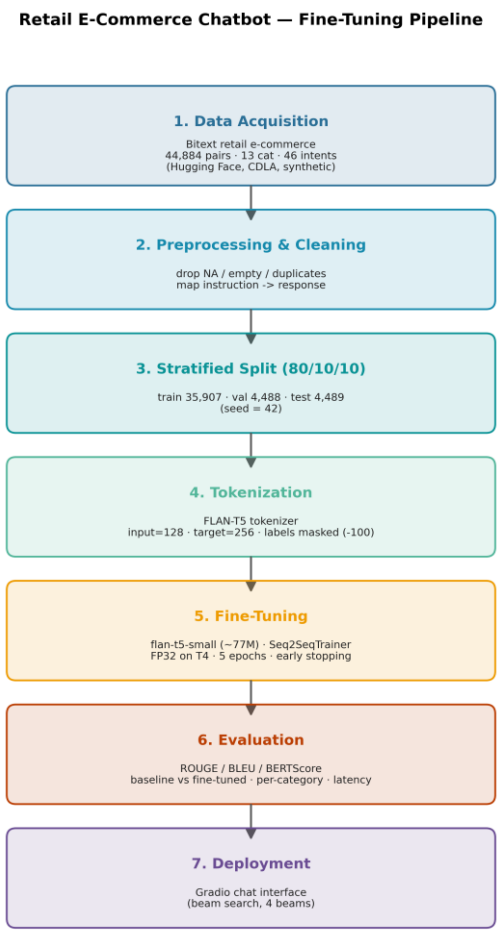

Fig. 8. End-to-end fine-tuning pipeline (data acquisition → preprocessing → stratified split → tokenization → fine-tuning → evaluation → deployment).

Code / Reproducibility. Full code: https://github.com/Harish-or-Peter/Retail-Ecommerce-Chatbot . The repository contains the end-to-end Colab notebook, tokenizer/config, paper draft, figures, and metrics; the ~293 MB fine-tuned weights are regenerated by running the notebook, and the dataset auto-downloads from Hugging Face.